

اصل (Single Responsibility Principle) SRP می‌گه:

هر کلاس باید تنها یک دلیل برای تغییر کردن داشته باشه. یعنی کلاس فقط یک وظیفه‌ی مشخص داشته باشه.

اگه یک کلاس چند مسئولیت رو همزمان به عهده بگیره:

نگهداری و تغییرش سخت میشه.

تست‌نویسی براش مشکل میشه.

در آینده تغییرات در یک بخش ممکنه ناخواسته بخش دیگه رو خراب کنه.

✗ مثال نقض SRP در C#

```
public class Invoice
{
    public void CalculateTotal()
    {
        // محاسبه جمع فاکتور
        Console.WriteLine("Calculating total price...");
    }

    public void SaveToDatabase()
    {
        // ذخیره در دیتابیس
        Console.WriteLine("Saving invoice to database...");
    }

    public void PrintInvoice()
    {
        // چاپ فاکتور
        Console.WriteLine("Printing invoice...");
    }
}
```

● مشکل:

این کلاس هم مسئول محاسبه‌ی جمع فاکتور هست،

هم ذخیره‌سازی در دیتابیس،

هم چاپ کردن.

یعنی سه دلیل برای تغییرش وجود داره (تغییر در قوانین مالی، تغییر در دیتابیس، تغییر در نحوه چاپ). این نقض اصل SRP هست.

✓ اصلاح مطابق SRP

```
// فقط محاسبه و منطق فاکتور
public class Invoice
{
    public void CalculateTotal()
    {
        Console.WriteLine("Calculating total price...");
    }
}

// فقط مسئول ذخیره‌سازی
public class InvoiceRepository
{
    public void SaveToDatabase(Invoice invoice)
    {
        Console.WriteLine("Saving invoice to database...");
    }
}

// فقط مسئول چاپ
public class InvoicePrinter
{
    public void Print(Invoice invoice)
    {
        Console.WriteLine("Printing invoice...");
    }
}
```

● حالا:

اگر قوانین محاسبه تغییر کرد، فقط کلاس Invoice تغییر می‌کند.

اگر دیتابیس عوض شد، فقط InvoiceRepository.

اگر پرینت تغییر کرد، فقط InvoicePrinter.